

PYTHON을 이용한 데이터 분석 · 11강 실습

```
model = KMeans()  
model.fit(X_scaled)  
K=3, Silhouette=0.65
```

파이썬으로

K-means 클러스터링 구현하기

실습편 (2시간) — 고객 세분화 데이터로 직접 클러스터링을 해보고 비즈니스 인사이트를 찾아보자

2시간의 여정을 한눈에

Part 0	환경 설정 (12분)	라이브러리 import, 데이터 로드 및 검증
Part 1	데이터 탐색 (18분)	통계 요약 분석, 분포 및 관계 시각화
Part 2	전처리 (15분)	StandardScaler 정규화 (필수!)
Part 3	K-means (20분)	초기화 → 할당 → 업데이트 → 수렴
Part 4	최적 K (18분)	엘보우 / 실루엣 / 최종 결정
Part 5	해석 (17분)	클러스터 시각화 및 비즈니스 인사이트

환경 설정 체크리스트

1

Import 확인

`numpy / pandas / sklearn.preprocessing / sklearn.cluster / matplotlib`

2

데이터 로드

`df = pd.read_csv('customer_data.csv')`

3

df 구조 확인

`df.head() / df.shape / df.info()`

4

컬럼명 확인

`[age, income, spend_score, loyalty]` 4개

5

결측치 확인

`df.isnull().sum()` — 없어야 함

데이터 로드 및 기본 확인

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

df = pd.read_csv('customer_data.csv')
print(df.head()) # 첫 5줄 확인
print(df.info()) # 데이터 타입 및 결측치
print(df.describe()) # 통계 요약

X = df[['age', 'income', 'spend_score', 'loyalty']]
print(f'Shape: {X.shape}') # (300, 4) 예상
```

데이터 탐색 — 고객 세분화 데이터셋

 고객 300명의 나이, 연소득, 소비점수, 충성도로 세 가지 타입을 자동으로 찾기

df.describe() 예상 결과

	age	income	spend
mean	45.32	52,400	0.55
std	15.27	18,200	0.28
min	18.0	12,000	0.00
max	70.0	90,000	1.00

다음 단계:

각 컬럼의 분포를 히스토그램으로 확인

두 특성씩 scatter plot

데이터 시각화 — 분포 및 관계 파악

```
# 4개 특성의 분포 확인
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0,0].hist(df['age'], bins=20, color='teal', edgecolor='black')
axes[0,0].set_title('Age Distribution')

# 두 특성 간 관계 (Scatter plot)
axes[0,1].scatter(df['age'], df['spend_score'], alpha=0.6)
axes[0,1].set_title('Age vs Spend Score')

# 상관계수 행렬 (어떤 특성이 밀접한가?)
corr = df.corr()
print(corr) # 상관계수 확인
```

데이터 전처리 — StandardScaler 필수!

⚠️ 특성의 범위가 다르면 거리 계산이 왜곡됩니다!

특성	최소~최대	범위
age	18 ~ 70	52
income	12,000 ~ 90,000	78,000
spend_score	0 ~ 1	1

❌ 문제: Income의 범위(78,000)가 다른 특성을 완전히 지배!

거리 계산 예시 (스케일 없음)

고객 A: [25, 30000, 0.4]

고객 B: [26, 32000, 0.41]

$$\begin{aligned} \text{거리} &= \sqrt{(1)^2 + (2000)^2 + (0.01)^2} \\ &\approx 2000 \text{ (income이 지배!)} \end{aligned}$$

✓ 해결: StandardScaler 사용

StandardScaler 적용

```
from sklearn.preprocessing import StandardScaler

# 1단계: Scaler 객체 생성
scaler = StandardScaler()

# 2단계: 데이터 정규화 (평균=0, 표준편차=1)
X_scaled = scaler.fit_transform(X)

# 결과 확인
print(X_scaled.mean(axis=0)) # 모두 0 근처
print(X_scaled.std(axis=0))  # 모두 1 근처

# X_scaled는 (300, 4) 모양이지만 범위가 모두 비슷함
```


K-means Step 1: 초기화

K개의 중심점을 무작위로 선택합니다. (K=3 가정)

```
# 손으로 계산 (원리 이해용)
c1 = X_scaled[5]      # 무작위 선택
c2 = X_scaled[102]
c3 = X_scaled[250]

centers = [c1, c2, c3]
print('초기 중심점:')
for i, c in enumerate(centers):
    print(f'C{i}: {c}')

# 이 초기값이 최종 결과에 영향을 줌!
```

💡 sklearn의 해결책

n_init=10으로 설정하면, 10번 다른 초기값으로 실행한 후 가장 좋은 결과를 자동으로 선택합니다!

```
model = KMeans(n_clusters=3, n_init=10,
               random_state=42)
```

random_state=42는 결과 재현을 위해 필수!

K-means Step 2~4: 할당 → 업데이트 → 반복

```
# sklearn으로 자동 실행
model = KMeans(n_clusters=3, random_state=42, n_init=10)
model.fit(X_scaled) # 내부적으로 4단계를 반복 실행

# 결과 추출
labels = model.labels_ # 각 고객의 클러스터 번호 (0, 1, 2)
centers = model.cluster_centers_ # 최종 중심점 3개
inertia = model.inertia_ # SSE (작을수록 좋음)

# 몇 번 반복했는지 확인
print(f'수렴까지 반복: {model.n_iter_} 회')
print(f'Inertia: {inertia:.2f}')
print(f'클러스터 분포: {np.bincount(labels)}')
```

최적 K 찾기 — 엘보우 (Elbow) 방법

K=1부터 K=8까지 각각 K-means를 실행해서 inertia를 계산하고 그래프를 그립니다.

```
inertias = []
K_range = range(1, 9)

for k in K_range:
    m = KMeans(n_clusters=k,
               random_state=42,
               n_init=10)
    m.fit(X_scaled)
    inertias.append(m.inertia_)

plt.plot(K_range, inertias, 'o-')
plt.xlabel('K')
plt.ylabel('Inertia (SSE)')
plt.title('Elbow Method')
plt.show()
```

그래프 해석

K가 증가할수록 inertia는 감소하지만, 어느 K부터는 감소 속도가 급격히 느려집니다.

그 "꺾인 점"이 최적의 K입니다! (팔꿈치 모양으로 보인다고 해서 elbow)

예: 보통 K=2, K=3, K=4 중에서 선택됩니다.

최적 K 찾기 — 실루엣 점수 (Silhouette)

각 K에 대해 실루엣 점수를 계산합니다. (높을수록 좋음: -1 ~ 1)

```
from sklearn.metrics import silhouette_score

silhouettes = []
K_range = range(2, 9)  # K≥2

for k in K_range:
    m = KMeans(n_clusters=k,
               random_state=42)
    m.fit(X_scaled)
    score = silhouette_score(
        X_scaled, m.labels_)
    silhouettes.append(score)

best_k = K_range[np.argmax(silhouettes)]
print(f'최적 K: {best_k}')
```

점수 해석

0.8 ~ 1.0: 매우 좋음 (명확한 클러스터)

0.5 ~ 0.8: 좋음 (적당히 구분됨)

0.25 ~ 0.5: 약함 (겹침 많음)

< 0.25: 의심 (클러스터링이 적절하지 않음)

최적 K로 재학습 및 클러스터 시각화

```
# 엘보우/실루엣에서 결정한 K (예: K=3)로 재학습
model_final = KMeans(n_clusters=3, random_state=42, n_init=10)
model_final.fit(X_scaled)

# 클러스터 라벨을 원본 데이터프레임에 추가
df['cluster'] = model_final.labels_

# 2D 시각화: 두 개 특성 기준으로 그리기
plt.scatter(df['age'], df['spend_score'],
            c=df['cluster'], cmap='viridis', alpha=0.6, s=50)

# 최종 중심점 표시 (큰 x 마크)
centers_original = scaler.inverse_transform(model_final.cluster_centers_)
plt.scatter(centers_original[:, 0], centers_original[:, 1],
            marker='X', s=200, c='red', edgecolors='black', linewidth=2)
plt.title('K-means Clustering Result (K=3)')
```

클러스터 해석 — 각 그룹의 특성 분석

각 클러스터의 평균을 계산해서 그룹 특성을 파악합니다.

```
# 클러스터별 평균 계산
for i in range(3):
    cluster_data = df[
        df['cluster'] == i
    ]
    print(f'\nCluster {i}:')
    print(cluster_data[
        ['age', 'income',
         'spend_score']
    ].mean())
    print(f'크기: {len(cluster_data)}명')
```

C0 절약형 청년

낮은 연소득, 낮은 소비

각 클러스터별 마케팅 전략

클러스터 0 절약형 청년

특성

특성: 저소득, 낮은 소비

전략

전략: SNS 광고, 할인 프로모션, 가성비 상품

클러스터 1 럭셔리 추구

특성

특성: 높은 소비, 프리미엄 선호

전략

전략: 프리미엄 상품 추천, VIP 이벤트

클러스터 2 안정형 중년

특성

특성: 안정적 소비, 높은 충성도

전략

전략: 충성도 프로그램, 재구매 유도

CHALLENGE 1

초기값 의존성 검증

TODO: random_state를 여러 값(0, 1, 2, 5, 10, 99)으로 바꿔가며 K-means를 실행하세요. 각 random_state에서 나온 inertia 값을 비교하세요.

```
inertias_random = []
for rs in [0, 1, 2, 5, 10, 99]:
    m = KMeans(n_clusters=3, random_state=rs, n_init=1) # n_init=1로 설정!
    m.fit(X_scaled)
    inertias_random.append(m.inertia_)

print('Inertia by random_state:')
for rs, inert in zip([0,1,2,5,10,99], inertias_random):
    print(f'RS={rs}: {inert:.2f}')
```

💡 발견: n_init=1로 하면 첫 초기값에만 의존하므로 결과가 다릅니다. sklearn은 n_init=10으로 자동 해결!

CHALLENGE 2

StandardScaler의 필수성

스케일링 없이 vs 스케일링 후 K-means 결과를 비교합니다.

```
# 스케일링 없음
model_no = KMeans(n_clusters=3,
                  random_state=42)
model_no.fit(X) # 원본 데이터!
inertia_no = model_no.inertia_

# 스케일링 적용
model_yes = KMeans(n_clusters=3,
                  random_state=42)
model_yes.fit(X_scaled)
inertia_yes = model_yes.inertia_

print(f'Inertia (no scale): {inertia_no}')
print(f'Inertia (scaled): {inertia_yes}')
```

결과 해석:

✗ 스케일 X: income이 지배해서 age와 spend_score는 무시됨. 클러스터 분포가 비현실적.

✓ 스케일 O: 모든 특성이 공정하게 작용. 합리적인 클러스터 분포.

결론: StandardScaler는 선택이 아니라 필수!

CHALLENGE 3

K값에 따른 클러스터 변화 시각화

TODO: K=2, K=3, K=4, K=5로 각각 K-means를 실행한 후, 결과를 4개 subplot으로 시각화하세요. K가 커질수록 클러스터가 어떻게 변하는지 관찰하세요.

```
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for idx, k in enumerate([2, 3, 4, 5]):
    m = KMeans(n_clusters=k, random_state=42)
    m.fit(X_scaled)
    axes[idx].scatter(df['age'], df['spend_score'], c=m.labels_, cmap='viridis', alpha=0.6)
    axes[idx].set_title(f'K={k}')
```

실습 완료 확인 사항

- | | | |
|---|--------------------------------|--|
| 1 | 데이터 로드 및 탐색 | <code>df.head()</code> , <code>df.describe()</code> 실행 및 분포 확인 |
| 2 | <code>StandardScaler</code> 적용 | <code>X_scaled = scaler.fit_transform(X)</code> 완료 |
| 3 | 엘보우 방법 | $K=1\sim 8$ inertia 계산 및 최적 K 결정 |
| 4 | 실루엣 점수 | 각 K 에 대한 <code>silhouette_score</code> 계산 |
| 5 | 최종 <code>K-means</code> | 최적 K 로 모델 학습 및 labels 추출 |
| 6 | 시각화 | 2D scatter plot으로 클러스터 표시 |
| 7 | 클러스터 해석 | 각 클러스터의 평균 특성 계산 및 설명 |
| 8 | 도전 문제 | 3가지 챌린지 완료 (초기값, 스케일, K 값 변화) |

오늘의 핵심

- Part 0** → 데이터 로드 및 기본 탐색
- Part 1** → 통계 요약 및 시각화로 데이터 이해
- Part 2** → StandardScaler로 정규화 (필수!)
- Part 3** → K-means 4단계 알고리즘 이해 및 실행
- Part 4** → 엘보우와 실루엣으로 최적 K 결정
- Part 5** → 클러스터 해석 및 비즈니스 전략 수립